```
1  -module(readers).
2  -export([readers/1]).
3
4  readers(N) ->
5    ets:new(tab, [public, named_table]),
6    Writer = fun() -> ets:insert(tab, {k, 42}) end,
7    Reader = fun(I) -> ets:lookup(tab, I), ets:lookup(tab, k) end,
8    spawn(Writer),
9    [spawn(fun() -> Reader(I) end) || I <- lists:seq(1, N)],
10   receive after infinity -> deadlock end.
```

Fig. 13. Writer-readers program in Erlang.

spawned on line 9 tries to read two entries from this table: some entry with a different key in each process (an integer in the range 1..N) and the entry keyed by `k`. The `receive` expression on line 10 forces the process executing the `readers` code to get stuck at this point, ensuring that the process owning the table stays alive, which in turn preserves the ETS table.

As mentioned, our implementation vehicle is Concuerror [Christakis et al. 2013], a stateless model checking tool for finding concurrency errors in Erlang programs or verifying their absence. The tool is publicly available at:

http://parapluu.github.io/Concuerror/

Given a program and a test to run, Concuerror uses a stateless search algorithm to systematically explore the execution of the test under conceptually all process interleavings. To achieve this, the tool employs a source-to-source transformation that inserts instrumentation at *preemption points* (i.e., points where a context switch is allowed to occur) in the code under execution. This instrumentation allows Concuerror to take control of the scheduler when the program is run, without having to modify the Erlang VM in any way. In the current VM, a context switch may occur at any function call. Concuerror inserts preemption points only at process actions that interact with (i.e., inspect or update) shared state. Concuerror supports the complete Erlang language and can instrument programs of any size, including any libraries they use.

For benchmarking purposes, we extended a previous version of Concuerror[1] with three DPOR algorithms: (i) the '*classic*' DPOR algorithm with the sleep set extension as presented by Flanagan and Godefroid [2005a], (ii) *source-DPOR*, and (iii) *optimal-DPOR*. To implement these we had to encode rules for dependencies between operations that constitute preemption points. These rules are shared between all DPOR variants. For lookups and inserts to ETS tables (i.e., reads and writes to shared data) the rules are standard (two operations conflict if they act on the same key and at least one is an insert). For sending and receiving operations the "happens before" relation ($\rightarrow_E$) is the following:

- Two `send`s are ordered by $\rightarrow_E$ if they send to the same process, even if the messages are the same. (Note that if we would not order two `send`s that send the same message, then when we reorder them, the corresponding `receive` will not "happen after" the same `send` statement.)
- A `send` "happens before" the `receive` statement that receives the message it sent. A race exists between these statements only if the `receive` has an `after` clause.

[1]The current (in 2016–2017) version of Concuerror supports the optimal-DPOR algorithm by default and the source-DPOR algorithm as an option. The 'classic' algorithm is no longer available as an option.